unit-3
impquestion for
mid

1. a) Define function. Explain function creation and function calling in python with an example program.

Definition :

A function is a block of code which performs a specific task. It can be reused any number of times.

Function creation :

A function is created using def keyword.

Syntax :

```
def function_name(parameters):
    statements
```

Function calling :

The function will be executed when it is called.

Example :

```
# function creation
def greet(name):
    print("Hello", name)
# function calling
greet("Sravya")
```

Output :
Hello Sravya

b) Explain local and global variables with suitable examples.

1) **Local variable** : A variable declared inside a function is called local variable. It can be used only within the function.

Example :

```
def display():
    x = 10        # local variable
    print(x)
display()
```

Output :
10

2) Global variable : A variable declared outside a function is called global variable. It can be used throughout the program.

Example :

```
x = 20            # global variable
def show():
    print(x)
show()
```

Output :
20

Difference between Local and Global variables

Local variable	Global variable
1. Declared inside a function	1. Declared outside a function
2. Used only within the function	2. Used throughout the program
3. Temporary in nature	3. Available till the end of program
4. Cannot be accessed outside the function	4. Can be accessed inside any function

2. a) Explain different function arguments with suitable examples.

1) Positional arguments : Arguments are passed in correct order.

Example :

```
def add(a, b):
    print(a + b)
add(2, 3)
```

Output :
5

2) Keyword arguments : Arguments are passed using parameter names.

Example :

```
def student(name, age):
    print(name, age)
student(age=18, name="Ravi")
```

Output :
Ravi 18

3) Default arguments : A default value is assigned to parameter.

Example :

```
def greet(name="Guest"):
    print("Hello", name)
greet()
```

Output :
Hello Guest

4) Variable length arguments : Number of arguments is unknown.

Example :

```
def total(*nums):
    s = 0
    for n in nums:
        s = s + n
    print(s)
total(1, 2, 3, 4)
```

Output :
10

b) Write a python function that takes two lists and returns True if they have at least one common member.

```
def common_data(list1, list2):
    for x in list1:
        for y in list2:
            if x == y:
                return True
    return False

list1 = [1, 2, 3, 4]
list2 = [5, 6, 3, 8]
print(common_data(list1, list2))
```

Output :
True

3. a) Explain anonymous functions with examples.

Anonymous functions are the functions without a name. They are created using lambda keyword.

Syntax : lambda arguments : expression

Example 1 :

```
square = lambda x : x * x
print(square(5))
```

Output :
25

Example 2 :

```
add = lambda a, b : a + b
print(add(3, 4))
```

Output :
7

b) Explain following keywords with suitable examples.

i) **import** : Used to import entire module.

Example :

```
import math
print(math.sqrt(25))
```

Output :
5.0

ii) **from** : Used to import specific functions from module.

Example :

```
from math import sqrt
print(sqrt(16))
```

Output :
4.0

iii) **as** : Used to give alias name to module.

Example :

```
import math as m
print(m.factorial(5))
```

Output :
120

4. a) List and explain standard modules in python.

Python provides many built-in modules.

Module	Purpose
1. math	Mathematical operations
2. random	Generate random numbers
3. datetime	Date and time operations
4. os	Operating system operations
5. sys	System-related information
6. statistics	Mathematical statistics
7. json	Work with JSON data
8. re	Regular expressions
9. csv	Read and write CSV files
10. time	Time related functions

Example (random module) :

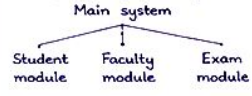
```
import random
print(random.randint(1, 10))   # random number between 1 and 10
```

b) Differentiate top-down and bottom-up approaches with examples.

Top-Down Approach	Bottom-Up Approach
1. Starts from the main problem	1. Starts from smaller modules
2. Divides problem into subproblems	2. Combines modules into system
3. Easy for planning	3. Easy for testing and debugging
4. Used in structured design	4. Used in object-oriented design

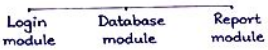
Example :

Top-Down Approach
(College Management System)



Bottom-Up Approach

First create small modules



Then combine all to form complete system.

5. a) Explain the advantage of packages. Explain the procedure to create a package and use the same.

Advantages of packages :

1. Organizes modules in a proper way.
2. Avoids name conflicts.
3. Reusability of code.
4. Easy maintenance.

Procedure to create a package :

Step 1 : Create a folder (mypackage)

```
mypackage /
```

Step 2 : Create __init__.py file in that folder.

```
mypackage /
    __init__.py
```

Step 3 : Create a module file in that folder.

```
mypackage /
    __init__.py
    add.py

# add.py
def addition(a, b):
    return a + b
```

Step 4 : Use the package.

```
from mypackage.add import addition
print(addition(2, 3))
```

Output :
5

b) Define Recursion. Explain Recursive functions with suitable examples.

Definition : Recursion is a process in which a function calls itself directly or indirectly.

A function that calls itself is called recursive function.

Example : Factorial of a number

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Output :
120

Explanation :

```
factorial(5)
= 5 * factorial(4)
= 5 * 4 * factorial(3)
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1
= 120
```

unit-4
imp questions

1. a) Define inheritance and explain types of inheritance with examples.

Inheritance is a mechanism in OOPs that allows a class (child) to inherit attributes and methods from another class (parent). It promotes code reusability.

Types of Inheritance

1) Single Inheritance
One child class inherits from one parent class.

```
class A:
    def showA(self):
        print("Class A")

class B(A ):

    pass

b = B()
b.showA()      # Output: Class A
```

2) Multiple Inheritance
One child class inherits from multiple parent classes.

```
class A:
    def showA(self):
        print("A")
class B:
    def showB(self):
        print("B")
class C(A, B):
    pass

c = C()
c.showA()     # A
c.showB()     # B
```

3) Multilevel Inheritance
A class inherits from a derived class.

```
class A:
    def showA(self):
        print("A")
class B(A):
    def showB(self):
        print("B")
class C(B):
    def showC(self):
        print("C")
c = C()
c.showA();c.showB();c.showC()   # A B C
```

4) Hierarchical Inheritance
Multiple child classes inherit from the same parent class.

```
class A:
    def show(self):
        print("Parent")

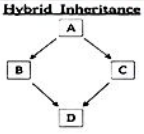
class B(A): pass

class C(A): pass

b = B(); c = C()

b.show()   # Parent
c.show()   # Parent
```

5) Hybrid Inheritance (combination)




```
class A:  pass
class B(A): pass
class C(A): pass
class D(B, C): pass
```

1. b) Demonstrate Polymorphism using an example Python program.

Polymorphism means "many forms". It allows the same method name to behave differently for different objects.

```
class Animal:
    def speak(self):
        pass

class Dog(Animal):
    def speak(self):
        return "Bark"

class Cat(Animal):
    def speak(self):
        return "Meow"

for a in [Dog(), Cat()]:
    print(a.speak())
```

Output :
```
Bark
Meow
```

2. a) Demonstrate operator overloading with an example program.

Operator overloading allows a class to give special meaning to operators.

```
class Complex:
    def __init__(self, real, img):
        self.real = real
        self.img = img
    def __add__(self, other):
        r = self.real + other.real
        i = self.img + other.img
        return Complex(r, i)
    def display(self):
        print(f"{self.real} + {self.img}i")

c1 = Complex(2, 3)
c2 = Complex(4, 5)
c3 = c1 + c2
c3.display()
```

Output :
```
6 + 8i
```

2. b) Demonstrate file operations with a suitable Python program.

```
# File operations
# 1. Write
f = open("sample.txt", "w")
f.write("Hello Python")
f.close()

# 2. Read
f = open("sample.txt", "r")
data = f.read()
print(data)   # Hello Python
f.close()

# 3. Append
f = open("sample.txt", "a")
f.write("\nFile Handling")
f.close()
```

Contents of sample.txt
```
Hello Python
File Handling
```

3. a) Demonstrate the design with classes using a case study.

Case Study : Library Management System (Design with classes)

```
class Book:
    def __init__(self, id, title, author):
        self.id = id
        self.title = title
        self.author = author
    def display(self):
        print(self.id, self.title, self.author)

class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
    def show_books(self):
        for b in self.books:
            b.display()

b1 = Book(101, "Python", "Guido")
b2 = Book(102, "Data Science", "Tom")
lib = Library()
lib.add_book(b1)
lib.add_book(b2)
lib.show_books()
```

Output :
```
101 Python Guido
102 Data Science Tom
```

3. b) Explain the functions write and write lines with examples.

write() : Writes a string to the file.

```
f = open("data.txt", "w")
f.write("First Line")
f.close()
```

write() output
```
First Line
```

writelines() : Writes a list of strings to the file.

```
lines = ["Line1\n", "Line2\n", "Line3\n"]
f = open("data.txt", "w")
f.writelines(lines)
f.close()
```

writelines() output
```
Line1
Line2
Line3
```

4. a) Demonstrate the log file writing in Python program.

```
import logging
logging.basicConfig(filename="app.log",
                    level=logging.INFO,
                    format="%(asctime)s - %(levelname)s - %(message)s")
logging.info("Application started")
logging.warning("This is a warning")
logging.error("This is an error")
logging.info("Application ended")
```

Contents of app.log
```
2024-05-26 10:00:11 - INFO - Application started
2024-05-26 10:00:12 - WARNING - This is a warning
2024-05-26 10:00:13 - ERROR - This is an error
2024-05-26 10:00:14 - INFO - Application ended
```

4. b) Develop a program to read Config files in Python.

```
import configparser
config = configparser.ConfigParser()
config.read("config.ini")

host = config["Database"]["host"]
user = config["Database"]["user"]
password = config["Database"]["password"]

print("Host:", host)
print("User:", user)
print("Password:", password)
```

config.ini
```
[Database]
host = localhost
user = admin
password = admin123
```

Output :
```
Host: localhost
User: admin
Password: admin123
```

5. a) Demonstrate read, readline, and readlines with examples.

1) read() : Reads entire file.

```
f = open("file.txt", "r")
data = f.read()
print(data)
f.close()
```

2) readline() : Reads one line at a time.

```
f = open("file.txt", "r")
line1 = f.readline()
line2 = f.readline()
print(line1, end=" ")
print(line2, end=" ")
f.close()
```

3) readlines() : Reads all lines and returns list.

```
f = open("file.txt", "r")
lines = f.readlines()
for line in lines:
    print(line, end=" ")
f.close()
```

Contents of file.txt
```
Python is easy.
It is powerful.
File handling is important.
```

5. b) Describe the following terms in detail and depict an example reflecting all.

i) **self** : Reference to the current instance of the class.

ii) **Constructor** : Special method __init__() called automatically when object is created.

iii) **Class variable** : Variable declared inside class but outside methods. Shared by all objects.

iv) **Instance variable** : Variable declared inside __init__(). Each object has its own copy.

```
class Demo:
    count = 0              # class variable
    def __init__(self, name):   # constructor
        self.name = name        # instance variable
        Demo.count += 1
    def display(self):
        print("Name:", self.name)
        print("Count:", Demo.count)
d1 = Demo("Ravi")
d2 = Demo("Anu")
d1.display()
d2.display()
```

Output :
```
Name: Ravi
Count: 2
Name: Anu
Count: 2
```

6. a) List and explain object-oriented concepts.

1) Class : Blueprint or template for objects.
2) Object : Instance of a class.
3) Encapsulation : Binding data and methods into a single unit.
4) Abstraction : Hiding internal details, showing essential features.
5) Inheritance : Deriving new class from existing class.
6) Polymorphism : One interface, multiple implementations.

6. b) What are the different modes of opening a file in Python? Explain the Python 'open()' built-in function.

Mode	Description
r	Open for read (default). File must exist.
w	Open for write. Creates new file or truncates existing.
a	Open for append. Adds at the end.
x	Create a new file. Fails if file exists.
b	Binary mode.
t	Text mode (default).
+	Open for updating (read and write).
rb, wb, ab, r+, w+, a+	Combinations of above modes.

open() syntax : open(filename, mode, buffering=-1, encoding=None)

- filename : name of the file
- mode : mode of opening
- Returns a file object

7. a) Write a Python program to create a class 'student' with members [name, branch, grade]. Define appropriate member functions for reading and displaying the student information.

```
class Student:
    def __init__(self):
        self.name = ""
        self.branch = ""
        self.grade = ""
    def read(self):
        self.name = input("Enter name: ")
        self.branch = input("Enter branch: ")
        self.grade = input("Enter grade: ")
    def display(self):
        print("Name:", self.name)
        print("Branch:", self.branch)
        print("Grade:", self.grade)

s = Student()
s.read()
s.display()
```

Input/Output Example
```
Input:
  Enter name: Ravi
  Enter branch: CSE
  Enter grade: A

Output:
  Name: Ravi
  Branch: CSE
  Grade: A
```

7. b) Write Python code snippets to illustrate Method overriding and Method overloading concepts.

Method Overriding (Polymorphism with inheritance)

```
class Parent:
    def show(self):
        print("Parent show")
class Child(Parent):
    def show(self):
        print("Child show")

c = Child()
c.show()     # Output: Child show
```

Method Overloading (Same method name with different parameters)

```
class Calc:
    def add(self, a, b=None, c=None):
        if c is not None:
            return a + b + c
        elif b is not None:
            return a + b
        else:
            return a
obj = Calc()
print(obj.add(5))          # 5
print(obj.add(5, 10))      # 15
print(obj.add(5, 10, 15))  # 30
```

unit-5
imp questions

1. a) List and explain any four built-in error types in Python.

Error Type	Explanation & Example

1. ZeroDivisionError	Raised when a number is divided by zero.
	`a = 10`
	`b = 0`
	`print(a/b)  # ZeroDivisionError`

2. NameError	Raised when a variable name is not found in the local or global scope. `print(x)  # NameError`
3. TypeError	Raised when an operation or function is applied to an object of inappropriate type. `a = 5 + "Hello"  # TypeError`

4. IndexError	Raised when a sequence subscript is out of range.
	`lst = [10, 20, 30]`
	`print(lst[5])  #IndexError`

1. b) Compare terminal based with GUI-based programs.

Feature	Terminal-Based Programs	GUI-Based Programs
Interface	Text based (command line)	Graphical (windows, icons, buttons)
User Interaction	Keyboard only	Mouse and keyboard
Ease of Use	Difficult for beginners	Easy and user friendly
Development	Less time and code	More time and code
Performance	Faster execution	Relatively slower
Memory Usage	Less memory	More memory
Portability	High	Depends on OS/Toolkit
Example	Python shell, cmd programs	Tkinter apps, desktop apps

2. a) Define exceptions and how to handle exceptions in a Python program?

- **Exception:** An exception is an error that occurs during the execution of a program and disrupts the normal flow.
- **Exception Handling:** It is the process of handling runtime errors using try and except blocks so that the program does not terminate abnormally.
- **Syntax:**

```
try:
    # risky code
except ExceptionType:
    # handling code
```

- **Example:**

```
try:
    a = 10
    b = 0
    print(a/b)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

2. b) Develop a Python program to handle division by zero exception.

```
try:
    num1 = int(input("Enter first number: "))
    num2 = int(input("Enter second number: "))
    result = num1 / num2
    print("Result =", result)
except ZeroDivisionError:
    print("Error! Division by zero is not allowed.")
```

3. a) Demonstrate GUI-based program coding with an example.

```
from tkinter import *
window = Tk()
window.title("Simple GUI Example")
label = Label(window, text="Welcome to Python GUI",
              font=("Arial", 14), fg="blue")
label.pack(padx=20, pady=20)
window.mainloop()
```



3. b) How to create, raise and handle user defined exceptions in Python? Illustrate with a Python program.

- **Create:** Create a class by inheriting Exception class.
- **Raise:** Use raise keyword to raise an exception.
- **Handle:** Use except block to handle the exception.

Program:

```
class AgeError(Exception):
    pass

try:
    age = int(input("Enter your age: "))
    if age < 18:
        raise AgeError("Age must be 18 or above")
    print("You are eligible to vote.")
except AgeError as e:
    print("Error:", e)
```

4. a) List and explain clean-up actions with examples.

Clean-up actions are the actions performed to release resources or reset the system after exception handling. These are written in **finally** block.

Common clean-up actions with examples:

Clean-up Action	Explanation	Example
Close a file	Closes the file after reading/writing	`file.close()`
Close DB connection	Closes the database connection	`conn.close()`
Release resources	Releases memory, locks, network connections	`del obj` `unlock()`
Print/Log message	Prints log or status message	`print("Clean up done")`

4. b) Develop a Python program to implement a single try block with multiple except blocks and trace the code for its execution.

Program:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result =", result)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input. Please enter numbers only.")
except Exception as e:
    print("Some other error:", e)
print("Program Ended")
```

Trace of Execution:

Case	Input (a, b)	Exception Raised	Output
1	10, 2	No Exception	Eneult = 5.0 Program Ended
2	10, 0	ZeroDivisionError	Cannot divide by zero Program Ended
3	10, 'x'	ValueError	Invalid input. Please enter numbers only. Program Ended
4	@, 5	ValueError	Invalid input. Please enter numbers only. Program Ended

5. a) Is it possible to implement multiple exception blocks in Python exception handling? Justify your answer.

Yes, it is possible.

- Python allows multiple **except** blocks in a single try block.
- Each **except** block handles a specific type of exception.
- It makes the program more specific and reliable.

Example:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print(a / b)
except ZeroDivisionError:
    print("Division by zero error")
except ValueError:
    print("Invalid input error")
```

5. b) Explain the purpose of 'else' and 'finally' blocks in exception handling with a Python program.

- **else block:** Executes when no exception occurs in **try** block.
- **finally block:** Executes always whether exception occurs or not.

Program:

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Result =", result)
finally:
    print("Execution completed")
```

Output:

Case 1: (10, 2)

```
Enter first number: 10
Enter second number: 2
Result = 5.0
Execution completed
```

Case 2: (10, 0)

```
Enter first number: 10
Enter second number: 0
Cannot divide by zero
Execution completed
```